

科创工作坊第三期——队列与栈

零. 写在前面

本次为了授课方便，演示时头文件直接用万能头文件`#include <bits/stdc++.h>`，并且直接使用了命名空间`using namespace std`。仅仅是为了写题时更方便，并不代表大家之后写学院作业及写工程时也要这样写，大家应当遵循作业/项目的格式要求。

为了方便，本次涉及字符串的部分均用`string`

本次可能涉及到的其他工具，不确定课上学过没有，但是与数据结构的主题无关，遂在下面列出，大家不知道也不必担心，主要是要掌握队列与栈的核心逻辑及写法。

1. `string`：字符串类型。读出/输出用`cin/cout`，遍历常使用`for(int i=0;i<(int)s.size();i++)`
2. `sqrt(x)`：对`x`开根号
3. `while`循环：`while(XXX)`，当`XXX`成立时其一直执行
4. `puts("XXX")`：用于输出字符串并自动换行，等价于`printf("XXX\n")`
5. `long long/ unsigned long long`：比`int`存更大的整数的数据类型，输入输出分别用`%lld`，`%llu`
- 6.

一. C++中的类 (class)

鉴于大家反馈大三讲数据结构时常用到C++中的类(class)但是没讲过其语法，于是本次科创工作坊先对C++中的类进行讲解。

1. 类(class)

C++ 在 C 语言的基础上增加了**面向对象编程**，C++ 支持面向对象程序设计。类是 C++ 的核心特性之一。类是一种用户**自定义**的数据类型，它是一种封装了**数据和函数**的组合。类中的数据称为**成员变量**，函数称为**成员函数**。类可以被看作是一种模板，可以用来创建具有相同属性和行为的多个对象。

```
class students{
public:
    double id;
    string name;
    double point;
    double get(){
        if(sqrt(point)*10 < 100) return sqrt(point)*10;
        else return 100.0;
    }
};
```

上面是一段类最基本的应用，定义了成员变量与成员函数

同时，类的一个重要特性是可以控制成员的属性（**private/public/protected**），从而进行权限的控制

(1) **public**：public属性下的成员变量与成员函数能够在类外引用，如：

```
class student{
public:
    double id;
    string name;
    double point;
    double get(){
```

```

        if(sqrt(point)*10 < 100) return sqrt(point)*10;
        else return 100.0;
    }
};

int main()
{
    student a;
    a.id=1;
    a.name="Milan";
    a.point=20;
    cout << a.get();
    return 0;
}

```

这里所有的成员都能在类外部（如主函数内）被直接调用。

(2) **private**: private属性下的函数只能被类内部自己调用，这样可以增强某些变量的隐私性与安全性，如：

这样可以有效地保护Milan同学的原始分，让Milan同学的原始分不能被调用，而是赋分后的分数。

```

class student{
private:
    double point;
public:
    double id;
    string name;
    double get(){
        if(sqrt(point)*10 < 100) return sqrt(point)*10;
        else return 100.0;
    }
    double set_point(double _point){
        point=_point;
    }
};

int main()
{
    student a;
    a.id=1;
    a.name="Milan";
    //a.point=20; 这一句话会导致编译错误，因为point是private的变量，只能在类内部被调用，不
    可以在主函数内被调用。
    a.set_point(20);
    cout << a.get();
    return 0;
}

```

(3) **protected**: protected属性介于private属性与public属性之间，protected属性下的变量只有本类以及子类的内部可以访问。

```

class student{
protected:
    string id; //存放学生id
};

```

```

class CollStu :public student { //公有继承
public:
    void test() {
        id = "1000"; //因为id为父类的protected属性，所以子类也可以访问
    }
};

int main() {
    Stu s;
    //s.id //id为protected属性，外部不可访问
}

```

2. *class*与*struct*的区别（浅显版）：

- (1) *class*有不同的成员变量属性，而*struct*没有
- (2) *class*能更方便地定义成员函数

二、栈 (stack)

1. 栈

栈是一种先进后出的数据结构。考虑一个容器，下端封死（成为栈底），上端可以用来放数与取数（成为栈顶），那这就是一个栈。因为容器内的数不能改变顺序，所以要从容器中取出数据就一定要从栈顶到栈底取数。

数组可以很好地模拟栈，数组的开头可以视为不动的栈底，只需要维护一个栈顶指针 *top*，便可实现一个基本的栈。

• 栈的定义

```

int st[100]; //栈
int top; //栈顶指针

```

注意：对栈顶指针 *top* 的定义不一样会导致实现不一样，一般对 *top* 有两种定义方式：

1. *top* 指向栈顶的**下一个**元素，即如果当前栈存到了 *st*[8]，此时 *top* 应该等于 9。
2. *top* 指向**栈顶**元素，即如果当前栈存到了 *st*[8]，此时 *top* 应该等于 8。

• 栈的判断空：

1. 对于 *top* 指向栈顶的下一个元素的定义，栈为空说明栈顶的"下一个"元素就是栈底，即

```

top==1 //如果从1开始存数

```

2. 对于 *top* 指向栈顶元素的定义，栈为空说明栈顶元素是空，即

```

top==0 //如果从1开始存数

```

• 栈的初始化

初始时栈时空栈，因此对于 *top* 指向栈顶的下一个元素的定义，*top* 初始为 1；对于 *top* 指向栈顶元素的定义，*top* 初始为 0。

• 栈的插入

1. 对于 *top* 指向栈顶元素的下一个元素的定义，插入一个元素的时候应该先把 *top* 指向的位置填入当前要填的数，再让 *top* ++

```
int x;
cin >> x;
st[top++]=x;//先st[top]=x, 再top++
```

2. 对于 top 指向栈顶元素的定义, 插入一个元素的时候应该先让 $top++$, 再在当前 top 指向的位置填入当前要填的数

```
int x;
cin >> x;
st[++top]=x;//先top++, 再st[top]=x;
```

- **栈的删除** (不是空栈)
这个两个定义都一样:

```
top--
```

- **栈顶元素的输出**

输出 $st[top - 1]$ 或 $st[top]$

Exercise :

[B3614 【模板】栈](#)

注意:

1. x 的范围不在 int 范围内, 需要定义为 *unsigned long long* 形式, 并用 $\%llu$ 输入输出
2. 一个方便的判断操作类型的办法是用 *string* 类型读入操作, 并直接判断 $s == "push"$

2. 栈的应用

(1) 括号序列的匹配

如何判断一个括号序列 (仅含 $()\{\}$) 是否是一个合法的括号序列? 比如 $\{[(\)]\}$ 就是一个合法的括号序列而 $\{\}$ 、 $[\]$ 则不是。栈是一个很好的维护括号序列的数据结构。

操作: 从左至右遍历整个括号序列。

如果是左括号就把它加入栈中, 如果是右括号就判断当前的栈顶是不是与之互补的左括号。

若是, 则说明它们应当匹配在一起, 把它们匹配并把栈顶元素弹出;

若不是, 则说明这个右括号找不到一个左括号与它匹配, 这个括号序列不合法。

最后, 判断栈是不是为空, 若不为空, 说明有左括号没匹配上, 亦不合法。

若以上不合法判断都不满足, 说明其合法。

以下是一个只含小括号和中括号序列的版本:

```
#include <bits/stdc++.h>
using namespace std;
const int N=150;
int n,st[N],top,cnt;
int fl[N];
string s;
int id(char x){
    if(x=='(') return 1;
    if(x==')') return -1;
    if(x=='[') return 2;
```

```

        if(x==']') return -2;
    }
    int main()
    {
        cin >> s;
        for(int i=0;i<(int)s.size();i++){
            int t=id(s[i]);
            if(t<0){
                for(int j=top;j>=1;j--){
                    if(id(s[st[j]])>0){
                        if(id(s[st[j]])==t){
                            fl[i]=fl[st[j]]++cnt;
                            top--;
                        }
                    }
                    break;
                }
            }
            else st[++top]=i;
        }
        for(int i=0;i<(int)s.size();i++){
            if(!fl[i]){
                puts("Faux");
                retur 0;
            }
        }
        for(int i=0;i<(int)s.size();i++){
            printf("%d ",fl[i]);
        }
        return 0;
    }
}

```

大家可以试着运行这一段代码，看一看具体的匹配关系。

Exercice:

[P1739 表达式括号匹配](#) 模版，需要判断当前如果非小括号直接*continue*

tips : 可能需要string的操作

[P1241 括号序列](#) 一道很好的题，需要大家在判断是否合法的基础上建立匹配关系（提示：栈中的元素可以存下标而非括号种类）

[括号画家](#) 加了一个枚举，有时间可以练习一下

(2) 判断入栈序列

[P4387 【深基15.习9】验证栈序列](#)

已知一个栈的入栈顺序，想要判断给定的出栈顺序是否存在。

利用好栈的先进后出特性，用一个栈维护题给的栈，若该栈栈顶是当前出栈序列的第一个，就令其出栈。

三、队列 (queue)

1. 队列

队列是一种先进先出的数据结构。考虑一列人在排队，我们总是把人插入到队的尾部（称为队尾），而排完队的人总是从队的前端（称为队头）离开。同样，插入数据和移除数据也是从队尾(*tail*)插入，从队头(*head*)移除。

数组同样可以很好地模拟队列，维护两个指针 *head* 和 *tail*，表示队头和队尾，便可很好地实现队列的插入与删除。

- 队列的定义

```
int head=1, tail=0, q[10010];
```

这里仍然有两种定义，一种是把tail指向队尾的下一个元素，一种是把tail指向当前队尾元素。而head一般就指向队头元素。由于本人习惯于写指向当前队尾元素的，于是下面的操作均以此为定义方式来介绍。

- 队列的判断空

若队列为空，则说明队头的指针比队尾的指针大（若相等的话说明还有一个元素 $q[head] = q[tail]$ ）因此应当是：

```
tail==head-1
```

- 队列的初始化

一开始队列为空，若从1开始存数的话，应当是：

```
head=1, tail=0;
```

- 队列的插入

```
int x;  
cin >> x;  
q[++tail]=x;
```

- 队列的删除

(1) 删除队尾元素：

```
tail--
```

(2) 不停删除队尾元素：

```
while(head<=tail && 满足删除条件) tail--;
```

- 队尾元素的输出：

```
cout << q[tail];
```

Exercise:

[B3616【模板】队列](#)

2. 队列的应用

队列的应用十分广泛，从简单地维护一个队列，到搜索、图论算法，再到单调队列、优先队列等扩展，比比皆是。但是限于大家对算法的了解，我们仅以下面的例题介绍队列的一些应用。

Exercice :

[P1540 \[NOIP 2010 提高组\] 机器翻译](#)

一道典型的先进先出的模拟题，很适合队列练手。

[P1296 奶牛的耳语](#)

需要对队尾元素进行不断删除的题目，需要深刻领悟队列的性质。

提示：可以将“相互交流”进行一步转化

注意：这道题可能需要先将数据从小到大排序

3. 双端队列

很显然，既然队列能从队尾插入元素，从队头删除元素，那么我们也可以维护一个“扩展版”的队列，它可以从队头或队尾插入或删除元素，这样它就构成了一个“双端队列”（*deque*）

Exercice :

[P7505 \[Wdsr-2.5\] 小小的填轮兵团](#) 很典型的双端队列的题,需要不断地删除元素。

注意：

1. 可能同样需要对数据进行排序
2. 数据类型为 *long long*

四、STL 的实现

C++的STL已经帮我们实现了 *stack*、*queue*与*deque*，我们如果不想手写的话直接调用也可以。

1. *stack*

- 栈的定义：

```
stack <int> st//默认初始为空栈
```

- 栈的插入：

```
int x;
cin >> x;
st.push(x);
```

- 栈的删除：

```
st.pop();
```

- 栈的判断空：

```
st.empty();//若是空栈返回1，否则返回0
```

- 栈的栈顶元素取出：

```
int x=st.top();
```

- 栈的大小：

```
st.size();
```

2. *queue*

- 队列的定义：

```
queue <int> q;//默认为空队列
```

- 队列的插入：

```
q.push(x);//从队尾插入
```

- 队列的删除：

```
q.pop();//从队首删除
```

- 队列的判断空：

```
q.empty();
```

- 队列的大小：

```
q.size();
```

- 队尾元素的取出：

```
int x=q.front();
```

3. *deque*

- 插入与删除：

```
dq.push_back(x);  
dq.push_front(x);  
dq.pop_back(x);  
dq.pop_front(x);
```

- 元素的取出：

```
int x=dq.front();  
int y=dq.back();
```


